

API Key Leakage via Public GitHub Repositories

response

Supply Chain / DevSecOps · **Critical** severity · SOC IR Playbook

Incident type	Credential Exposure - Source Code Leak
Severity	Critical
Priority	Critical
Detection sources	GitHub Secret Scanning Alerts, TruffleHog, Gitleaks, Cloud Provider Alerts, Bug Bounty Reports

Scenario

A developer accidentally commits and pushes API keys, cloud credentials or other secrets to a public GitHub repository. These secrets can be harvested by attackers (including bots that monitor GitHub) and used to access critical systems, cloud resources or third-party APIs.

MITRE ATT&CK

T1087, T1528, T1552.001

Preparation

Enable GitHub secret scanning Activate GitHub Advanced Security or third-party scanners (e.g., Gitleaks, TruffleHog) Use pre-commit hooks Integrate secret detection tools to prevent commits with secrets Rotate secrets regularly Use vaults (e.g., HashiCorp Vault, AWS Secrets Manager) and enforce key expiry policies Educate developers On secure coding practices and the dangers of pushing secrets Monitor public GitHub repos Use threat intel and GitHub APIs to continuously scan for exposed org assets

Detection & Analysis

Secret detected in commit Alert from GitHub, internal scan or external report Identify secret type and scope API key, cloud access key, DB password, token, etc. Correlate with owning user/repo Find the developer who committed it and determine if repo is public Analyse exposure window How long was it public? Any signs of usage (e.g., logs, rate limits breached)?

Containment

Immediately revoke the exposed secret Deactivate API key, token or credential from provider Restrict affected services If the secret granted broad access, disable dependent integrations or pipelines Alert developer and security team Notify for immediate validation and remediation Remove sensitive commit from history Use tools like git filter-branch, BFG or git rebase to scrub secrets

Eradication

Replace exposed keys with new ones Generate and distribute new keys securely via vault or secret manager Audit cloud/API logs Look for signs of abuse using the leaked key during its exposure period Validate GitHub repo hygiene Review commit history and remove any other sensitive information Block repo or mark private If it still contains risks or needs re-evaluation

Recovery

Restore service access using new secrets Confirm integrations and pipelines are working with rotated credentials Re-enable affected users or systems Once no unauthorised access is detected Monitor for abuse Set alerts on any suspicious use of revoked credentials across services Document impact and confirm clean repo state Ensure dev teams comply with updated policies

Lessons Learned & Reporting

Conduct RCA Why and how was the secret exposed? Human error, misconfigured Git tools, no scanning? Improve pre-commit pipelines Integrate Git hooks or CI/CD scanners to block such mistakes earlier Train development teams On secure software development lifecycle (SSDLC) and version control hygiene Update incident documentation Include key timelines, revoked credentials and mitigation efforts Report if necessary For breaches involving regulated data or third-party systems (GDPR, PDPA, PCI DSS, etc.)

Tools typically involved

GitHub Advanced Security (secret scanning); TruffleHog, Gitleaks, GitRob; HashiCorp Vault, AWS Secrets Manager, Azure Key Vault; SIEM (e.g., Sentinel, Splunk) and threat detection systems; Version control auditing (e.g., Git log parsing, commit reviewers)

Success metrics (SLA targets)

Revocation Time <15 minutes from detection Commit Cleanup Time <1 hour for critical secrets Secret Replacement & Reintegration <4 hours for production use Exposure Window Analysis Completion Within 24 hours Developer Acknowledgement of Policy 100% of involved devs within 2 business days